

Modelagem Geométrica: Representações Celulares, Ray Tracing Volumétrico e Dinâmica de Partículas em Tempo Real

Richard Elias Soares Viana
IMPA Tech - Graduação em Matemática Aplicada

Maio de 2026

Abstract

Este artigo detalha a implementação de um sistema robusto de modelagem geométrica bidimensional, integrando representações celulares uniformes e adaptativas (Quadrees) com motores de renderização e física. O trabalho aborda a transição de visualizações estáticas para um ambiente dinâmico onde funções implícitas arbitrárias definem obstáculos para um sistema de iluminação volumétrica via *Ray Tracing* e partículas físicas. Discutimos como a utilização de estruturas de aceleração espacial permite o processamento em tempo real de fenômenos complexos como sombras, reflexões baseadas em gradientes e decaimento de luz.

1 Introdução

A representação eficiente de formas geométricas é um pilar da computação gráfica. Enquanto modelos paramétricos são intuitivos, as formas implícitas oferecem vantagens únicas em testes de inclusão e operações booleanas. Este projeto explora a discretização dessas formas em grades, utilizando a *Quadtree* não apenas como modelo de visualização, mas como uma estrutura de dados de aceleração para consultas espaciais de alto desempenho.

2 Representações Celulares e Quadrees

2.1 Grades Uniformes e Classificação

Dada uma região Ω e uma função implícita $f(x, y)$, a discretização uniforme divide o domínio em células de tamanho fixo. O status de cada célula é determinado pela avaliação de f :

- **Status 0 (Fora):** $f(x, y) > 0$ em toda a célula.
- **Status 1 (Dentro):** $f(x, y) \leq 0$ em toda a célula.
- **Status 2 (Interseção):** A fronteira $f(x, y) = 0$ atravessa a célula.

2.2 Quadtree Adaptativa e Poda (Pruning)

A Quadtree otimiza o espaço subdividindo apenas células de interseção (Status 2). **Implementação Técnica:** A estrutura foi implementada em `js/adaptive.js` utilizando recursão para construção e um método de pós-processamento de poda. O algoritmo de poda verifica recursivamente: se um nó pai possui quatro filhos com o mesmo status (ex: todos Status 0), os filhos são descartados e o pai assume aquele status. Isso é vital para que o *Ray Tracer* não perca tempo processando subdivisões desnecessárias no vazio.

3 Ray Tracing e Iluminação Volumétrica

O motor de renderização foi projetado para simular uma lanterna omnidirecional.

3.1 Aceleração por AABB (Slab Method)

Para cada raio $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, a travessia na Quadtree utiliza o teste de interseção com caixas delimitadoras alinhadas aos eixos (AABB). Para um nó com limites $[x_{\min}, x_{\max}]$ e $[y_{\min}, y_{\max}]$, calculamos:

$$t_{x1} = \frac{x_{\min} - o_x}{d_x}, \quad t_{x2} = \frac{x_{\max} - o_x}{d_x}$$

O raio intercepta o nó se o intervalo $[t_{\min}, t_{\max}]$ resultante da interseção de todos os eixos for válido e positivo. Esta técnica permite que o sistema ignore instantaneamente milhares de pixels que não estão na trajetória da luz.

3.2 Iluminação de Dentro para Fora

Um desafio implementado foi a detecção de imersão. Ao iniciar o *Ray Tracing*, o sistema avalia $f(\mathbf{o})$. Se $f(\mathbf{o}) \leq 0$, a luz nasce "dentro" do objeto. O algoritmo de colisão é invertido: o raio viaja pelo interior (azul) e só colide quando tenta "sair" da forma ($f > 0$). Isso cria um efeito visual onde apenas o interior do objeto é iluminado por uma luz azulada, enquanto o exterior permanece em trevas.

3.3 Decaimento de Luz (Falloff)

A intensidade da luz não é linear. Implementamos uma curva de decaimento quadrática para suavidade visual:

$$I(t) = I_0 \cdot \left(1 - \frac{t}{R}\right)^{2.5}$$

Onde R é o `lightRange` ajustável pelo usuário. O resultado é um núcleo brilhante que se dissipa suavemente, simulando dispersão volumétrica.

4 Física de Partículas e Colisões

4.1 Cálculo de Gradientes para Reflexão

As partículas dinâmicas em `js/main.js` colidem com as formas implícitas. Para calcular a direção do "quique", aproximamos a normal $\mathbf{n}$ da superfície usando o gradiente da função implícita via diferenças finitas:

$$\nabla f \approx \left[\frac{f(x + \epsilon, y) - f(x - \epsilon, y)}{2\epsilon}, \frac{f(x, y + \epsilon) - f(x, y - \epsilon)}{2\epsilon} \right]$$

A velocidade é então refletida: $\mathbf{v}_{ref} = \mathbf{v} - 2(\mathbf{v} \cdot \mathbf{n})\mathbf{n}$.

4.2 Sombras Dinâmicas (Line-of-Sight)

Cada partícula verifica se está sendo atingida pela luz antes de ser desenhada. Ela lança um *shadow ray* de volta para a lanterna. Se a Quadtree detectar qualquer nó de interseção ou preenchido entre a partícula e a luz, a partícula entra em "sombra" e sua renderização é omitida.

5 Rasterização de Funções Customizadas

O sistema permite a entrada de *strings* como " $x*x + y*y - 0.5$ ". **Implementação:** Utilizamos o construtor `new Function('x', 'y', ...)` para transformar a entrada do usuário em código executável de alta performance. As coordenadas do Canvas são mapeadas para um domínio canônico $[-2, 2]$ para que funções matemáticas clássicas funcionem sem ajustes de escala manuais.

6 Esferas Geodésicas

Embora o foco tenha se deslocado para o motor 2D, a seção de esferas em `js/sphere.js` mantém o rigor do refinamento do icosaedro. A cada nível de refinamento, novas faces são geradas pela conexão dos pontos médios projetados na superfície esférica ($\|v\| = 1$), resultando em uma malha de triângulos quase equiláteros que minimiza distorções de mapeamento.

7 Conclusão

A integração da Quadtree com o Ray Tracing e a física implícita demonstra o poder das estruturas de subdivisão espacial. O que começou como uma simples grade estática evoluiu para um sistema capaz de processar centenas de partículas e milhares de raios de luz em tempo real, provando que a modelagem geométrica matemática é a base para simulações visuais complexas e eficientes.